

Session Transcript - Building `health\_agents`

A record of the conversation that built this project: a CLI healthcare assistant with a LangGraph pipeline (input guardrail -> query classifier -> summarization / synthesis / QnA agents), using Groq-hosted LLMs.

Note: raw tool output (shell commands, file diffs, test runs) is summarized rather than reproduced verbatim, to keep this readable. One value has been redacted below: a live Groq API key that was accidentally printed to the terminal during debugging.

1. Kickoff - project scaffold

User:

I want to build a 3 langgraph agents that takes user query from cli it can in multi line text, even .txt and pdf file should take in put all this formats. 3 agents are: one is summarization of health report, second is synthesis health report generation, and third is normal qna bot. And I want you to build exactly according to my architecture plan - first, user query must first go through input guardrails since it's a healthcare project - if it fails it should ask the user about a sensitive topic or any things. After that passes it should go to query contextualizer - there should be classification of what to do: summarize, synthesize data, or normal qna, and this should be done by an LLM. We will use Groq available models for this, I have its API key. For this, initialize project in this directory using uv, and create a CLAUDE.md file for this, before building the 3 agents - I will tell you one by one what needs to be done.

Assistant: Flagged that the working directory sat inside a much larger, unrelated git repo (later found to be rooted at the whole home directory) and that no git actions would touch it. Classified the request as architectural but scoped the immediate step down to just the scaffold. Asked clarifying questions (via a multiple-choice prompt):

- Groq model -> **llama-3.3-70b-versatile**
- CLI style -> **Rich + Typer**
- PDF parsing -> **pypdf**
- Project/package name -> **health_agents**

Presented a short scaffold design (uv project, config/LLM/CLI modules, `.env.example`, `.gitignore`, `CLAUDE.md`) and asked for approval before building.

2. Agent 1 - Summarization

User:

Let's start with agent 1, its summarization of health report. It should summarize the health report in 3 paragraphs, and if the document contains prompt injection or content from another domain it should not show in the summary. Also, which model is best for summarization, use that, and build this agent first.

Assistant: Ran the uv scaffold (`uv init`), added `langgraph`, `langchain-groq`, `typer`, `rich`, `pypdf`, `pydantic-settings`, `python-dotenv`), wrote `config.py`/`llm.py`, and built the summarization agent as its own LangGraph `StateGraph` (`graphs/summarization_graph.py`): a single node that wraps the report text in `<health_report>` tags and instructs the model to treat that content strictly as data, never as instructions - the defense against prompt injection or off-domain content embedded in an uploaded document. Wired into `health-agents summarize --file <path>` (or paste + Ctrl-D). Wrote unit tests using `langchain_core`'s `GenericFakeChatModel` to assert the exact messages sent to the LLM, without hitting the real API. 4/4 tests passing.

3. Agent 2 - Synthesis (interview -> PDF report)

User:

For health report synthesis generation it should ask multiple questions to the user - at least 6, up to 10, depending on the user. Don't just say "sufficient" - it should be real two-way communication. Then it should generate a health report as a PDF. Let's build this.

Assistant: Built two small graphs: `graphs/interview_graph.py` (one `decide` node that asks the LLM for the next question and whether it has enough information, returned as strict JSON - with code, not the LLM, enforcing the 6-10 bound) and `graphs/synthesis_graph.py` (writes the final structured report from the full Q&A transcript). Added `io/report_pdf.py` using `fpdf2` to render the report to a PDF. Wired into `health-agents synthesize`, which drives the actual turn-by-turn loop from the CLI rather than using LangGraph's `interrupt()` primitive - documented in `CLAUDE.md` why: code before an `interrupt()` call re-executes on every resume, which would re-invoke the LLM and could produce a different question than the one already shown to the user.

Hit and fixed a real `fpdf2` bug along the way: `multi_cell()`'s default `new_x` is `RIGHT` (not `LMARGIN` as commonly assumed), so stacking

`multi\_cell` blocks without resetting `new\_x="LMARGIN"` explicitly threw `"Not enough horizontal space to render a single character"` on the second block. 13/13 tests passing after the fix, including a PDF round-trip test (write, then read back with `pypdf` and assert the text is present).

4. Agent 3 - QnA

User:

The QnA agent should be based on the LLM with prompt engineering and guardrails for off-domain questions, with a detailed system prompt covering greeting through general QnA conversation - build this as the 3rd agent.

Assistant: Built `graphs/qna\_graph.py` - one `respond` node, called per turn with the full running `list[BaseMessage]` history threaded through by the CLI (same stateless-per-turn pattern as the interview agent, for the same interrupt-replay reason). All behavior lives in one detailed system prompt (`prompts/qna.py`): greets only when the conversation has no prior user message, restricts scope to health/wellness topics with an explicit off-domain refusal-and-redirect instruction, resists attempts to override its role, and includes emergency/safety disclaimers. Wired into `health-agents qna` (chat loop, type `exit` to quit). 16/16 tests passing.

5. Wiring it together - guardrail + classifier + `ask`

User:

Now, based on this build, [add the] query contextualizer and input guardrails and wire them so that user queries in the CLI first go through input guard, then pass or fail - if it passes it goes to the query contextualizer, [which routes it] into the 3 agents based on what it needs - it should decide accurately for user queries.

Assistant: Built `graphs/guardrail\_graph.py` (one `check` node; strict JSON `{"passed": bool, "message": str}`; fails only on genuinely unsafe, injection, or completely off-domain content - sensitive but legitimate health topics are explicitly instructed to pass; fails `*closed*` if its own response can't be parsed) and `graphs/classifier_graph.py` (one `classify` node routing to `summarize` / `synthesize` / `qna`; unparseable output defaults to the safest catch-all, `qna`). Refactored `summarize` / `synthesize` / `qna` into shared `_run_*` flow functions and added a new `ask` command that calls`

guardrail -> classifier -> the matching flow - the main entrypoint
matching the original architecture. Added end-to-end CLI integration
tests using Typer's `CliRunner` with a faked LLM. 26/26 tests passing.

6. Per-task models

User:

I have one thing - instead of one model, use a different model for different tasks.

Assistant: Replaced the single shared model with five independently-configurable settings (`config.py`,
`GROQ\_MODEL\_GUARDRAIL` / `\_CLASSIFIER` / `\_SUMMARIZE` / `\_SYNTHESIZE` / `\_QNA` in `.env`), split into a "fast" tier for the guardrail/classifier (single-shot, low-complexity, run on every query) and a "quality" tier for summarize/synthesize/qna (need real reasoning over clinical content). Made `get\_groq\_chat(model, ...)` require `model` explicitly
- no more implicit shared default - so every call site has to say which task's model it wants. 27/27 tests passing.

7. First live run

User:

Let's run this in CLI.

Assistant: Ran a live smoke test with the real Groq API key and hit two real issues, both fixed on the spot:

1. `.env` had a stray space (`GROQ\_API\_KEY= gsk\_...`) - harmless for the app (`python-dotenv` tolerates it) but broke a debugging `source .env` command, which in doing so **printed the real API key to the terminal**. That value is redacted here and was never committed to the repo (`.env` stays in `.gitignore`); the key value itself is not reproduced in this transcript.
2. The originally-chosen default models (`llama-3.3-70b-versatile` / `llama-3.1-8b-instant`) returned `404 model\_not\_found` for this specific Groq account/key - the documented model ids aren't universally available; access is gated per key. Queried `client.models.list()` against the real key and found it only has access to the `openai/gpt-oss-\*` family (plus a few niche models, including `openai/gpt-oss-safeguard-20b`, a safety/policy-classification-tuned model - a better fit for the guardrail than a generic chat model). Updated all five defaults to

models this account can actually use and re-verified live: `ask` routed a real headache question to `qna` and got a well-scoped answer with safety caveats; `summarize` produced a clean 3-paragraph summary of a pasted sample lab report; the interview `decide` node returned valid JSON from the live model.

8. How to run it

User:

How do I run this?

Assistant: Gave the run commands: `uv run health-agents ask` as the main entrypoint (guardrail -> classifier -> routed agent), plus the direct per-agent commands (`summarize`, `synthesize`, `qna`) for testing a specific agent without going through the classifier.

9. Test questions

User:

Give me questions to test, I am testing.

Assistant: Provided a categorized set of sample queries: ones expected to route to QnA, ones expected to route to Synthesize, one expected to route to Summarize (with a sample report to paste), queries expected to *fail* the guardrail (off-domain / prompt injection, to confirm it re-prompts instead of crashing), and sensitive-but-legitimate queries expected to *pass* (e.g. mental health, sexual health) rather than being wrongly rejected.

10. Bug found while testing - classifier misroute

User: **(shared a screenshot)** - a query intended to trigger the synthesis interview instead got a single large markdown-formatted response with fill-in-the-blank lab tables - the interview never started; it wasn't asked question-by-question as designed.

Assistant: Reproduced it. First confirmed the interview loop itself works correctly when `synthesize` is called directly (questions do come one at a time) - so the bug was in routing, not the interview logic. Traced it to the classifier: phrasing like *"I want a summary of what tests I should track for my cardiologist visit"* was misrouted to `qna`

because the word "summary" pattern-matched toward the `summarize` intent, even though no existing document was involved. Rewrote `prompts/classifier.py` so the explicit decision rule is "does a document already exist" rather than "does the word 'summary' appear," and added worked examples covering exactly this case. Verified 9/9 against the live model, including the query that broke before. 27/27 tests still passing.

11. This step - push to a private repo + export this transcript

User:

Push this in a new private [repo] and also export this session chat in PDF and add [it] in the repo, let's do this.

Assistant: Found that `new\_project` sat inside a much larger git repository rooted at the home directory, with unrelated history - pushing from there would have dragged all of that in. Initialized a dedicated git repository scoped to just this project folder, confirmed `.env` (containing the real API key) is excluded via `.gitignore` and not staged, wrote this transcript (redacting the leaked key), rendered it to PDF, and pushed both to a new private GitHub repository.